

# HSH-64: 面向轻量化语义检索的 64 位可学习语义哈希

Zhihao Wu

1938258545@qq.com

2026 年 7 月 31 日

## 摘要

语义哈希将高维连续嵌入映射为紧凑的二进制码，从而以极小的内存占用实现次线性的近似最近邻 (ANN) 检索。然而，传统的哈希方法相比暴力嵌入检索往往存在较大的召回率差距，尤其在码长受限于边缘设备部署时更为明显。本文提出 **HSH-64**，一种专门面向轻量化中文词表级语义检索的 64 位可学习语义哈希方案。HSH-64 采用 `feat(4) + sim(52) + abs(8)` 的结构化编码格式，其中 52 位语义相似码通过一个三阶段流程端到端学习：(1) 基于强教师模型的连续预训练，(2) 基于直通估计器 (STE) 的离散精调，以及 (3) 基于贪心比特翻转的召回导向后处理。为了降低在线搜索开销，我们进一步集成了自适应半径扩展的多索引哈希 (MIH)、非对称距离评分，以及使用轻量编码器进行粗排、使用强模型进行精排的两阶段检索架构。在包含 3,109 个词的中文词表上的实验表明，最佳单模型 (h512, 1.17 MB) 在纯 Hamming 空间中 Recall@10 达到 **0.7404**，而轻量化版本 (h256, 585 KB) 达到 **0.7382**；四个模型的小规模集成达到 **0.7724**。在 MIH 粗排后接 bge-large 精排的两阶段系统中，四模型集成 Recall@10 达到 **0.8970**。源代码与可复现脚本见 [https://github.com/Sauomore/Cabinet\\_hsh64](https://github.com/Sauomore/Cabinet_hsh64)。

## 1 引言

高维嵌入空间中的近似最近邻 (ANN) 检索是现代信息检索、推荐系统以及大语言模型增强的基础模块 (Jégou, Douze, and Schmid, 2011; Guo et al., 2020)。基于预训练语言模型 (如 BERT (Devlin et al., 2019) 和 sentence-transformers (Reimers and Gurevych, 2019)) 的稠密检索取得了显著的准确率提升，但浮点向量的暴力比较在大规模语料或延迟敏感的边缘设备上代价过高。

语义哈希 (Salakhutdinov and Hinton, 2009) 提供了一种有吸引力的替代方案：它将每个向量压缩为短二进制码，使相似度可以通过快速的位运算 (例如通过 CPU POPCNT 计算 Hamming 距离) 来近似。早期方法依赖线性投影，如 PCA 或谱哈希 (Weiss, Torralba, and Fergus, 2008)；更近的深度语义哈希方法 (Xia et al., 2014; Cao et al., 2015; Li, S. Wang, and Kang, 2016) 训练神经网络端到端地生成保留语义邻域结构的二进制码。尽管取得了进展，两个实际挑战仍然存在：

1. **召回率与效率的权衡。** 短码减少了存储和计算，但信息容量有限，导致相比完整嵌入的召回率下降。

2. **部署成本。** 最先进的检索往往依赖大型编码器模型（如 bge-large）进行推理，这不适合轻量化设备。

本文通过引入 HSH-64（Hierarchical Semantic Hashing，层次化语义哈希）来应对这两个挑战。核心洞察是：面向词表级检索的语义哈希可以利用三种结构特性：(i) 项目可按词性或语义类别进行粗粒度划分；(ii) 核心语义相似度信号可由适量二进制超平面捕获；(iii) 同一粗粒度簇内的项目可通过短完美哈希标识符高效消歧。我们的贡献包括：

- 提出了一种结构化 64 位编码 `feat(4) + sim(52) + abs(8)`，将词性特征、语义相似位和簇内完美哈希标识符分离。该设计对于硬件更加友好，且可解释。
- 设计了一个 **三阶段训练流程**——连续预训练、基于 STE 的离散精调、召回导向的贪心后处理——这些方法显著提升了 52 位语义码的质量。
- 为 HSH-64 配备了 **自适应多索引哈希 (MIH)** 粗排器、非对称距离评分，以及两阶段检索架构：在线使用小编码器，仅在训练或可选精排时使用强模型。
- 在 3,109 词中文词表基准上进行了广泛的实证研究，包括模型容量分析、后处理超参数搜索、集成策略比较和效率基准。单个轻量化模型（585 KB）在纯 Hamming 空间中达到 0.7382 Recall@10，最佳后处理模型达到 0.7404，集成模型达到 0.7724，完整两阶段系统达到 0.8970 Recall@10。

## 2 相关工作

### 2.1 语义哈希

早期无监督方法如局部敏感哈希（LSH）(Datar et al., 2004) 和谱哈希 (Weiss, Torralba, and Fergus, 2008) 通过随机投影或相似图上的谱分解构造二进制码。LSH 的关键局限在于随机超平面不利用嵌入空间的语义结构，导致给定码长下召回率欠佳。监督哈希 (Liu et al., 2012; Shen et al., 2015) 引入标签信息以提高判别力。深度哈希 (Xia et al., 2014; Cao et al., 2015) 使用神经网络直接将原始输入或嵌入映射到二进制码，通常优化成对相似度保持损失。我们的工作通过引入结构化多字段编码和专门针对召回导向优化的三阶段训练流程来扩展这一方向。

### 2.2 离散优化的直通估计器

由于符号/量化函数几乎处处梯度为零，训练深度哈希模型需要梯度估计器。直通估计器 (STE) (Bengio, Léonard, and Courville, 2013) 将梯度直接复制通过符号函数，在二值神经网络中广泛使用 (Courbariaux et al., 2016)。我们在第二阶段采用 STE，并辅以直接优化离散码而无须梯度近似的后处理阶段。

### 2.3 多索引哈希

MIH (Norouzi, Punjani, and Fleet, 2012) 将长二进制码切分为多个子串，并为每个子串构建倒排索引。给定查询，MIH 搜索枚举每个子串在小 Hamming 半径内的所有可能，然后合并

候选列表。自适应半径扩展在召回率与候选池大小之间取得平衡 (J. Wang et al., 2018)。我们采用 MIH 并引入新颖的基于候选池饱和度的自适应停止准则。

## 2.4 乘积量化与紧凑编码

乘积量化 (PQ) (Jégou, Douze, and Schmid, 2011) 将向量空间分解为低维子空间的笛卡尔积，并独立量化每个子空间。对于 64 位预算，PQ 通常使用 8 个 8 位子空间，需要为每个子空间学习码本。与 HSH-64 相比，PQ 在许多场景下提供更高的召回率，但代价是存储全分辨率码本并在候选重排阶段计算精确距离。HSH-64 以部分召回率换取极致的存储效率（每项 8 字节）和硬件加速比较 (POPCNT)。

## 2.5 检索中的知识蒸馏

将大教师模型蒸馏为紧凑学生模型是高效检索的常见做法 (Hinton, Vinyals, and Dean, 2015; Izacard and Grave, 2020)。我们两次应用这一思想：首先将 bge-large 教师嵌入蒸馏到小 MLP 投影器；其次将强 HSH 码蒸馏到仅需要 bge-small 输入的轻量学生模型。

## 3 问题形式化

设  $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$  为包含  $N$  个项目的词表，每个项目由轻量化编码器（如 bge-small）产生的连续嵌入  $\mathbf{x}_i \in \mathbb{R}^D$  表示。设  $\mathcal{T} = \{\mathbf{t}_1, \dots, \mathbf{t}_N\}$  为强模型（如 bge-large）产生的对应教师嵌入。项目  $i$  与  $j$  之间的余弦相似度为

$$s_{ij} = \frac{\mathbf{t}_i^\top \mathbf{t}_j}{\|\mathbf{t}_i\| \|\mathbf{t}_j\|}. \quad (1)$$

项目  $i$  的真实 top- $K$  邻居集合为

$$\mathcal{G}_i^{(K)} = \arg \max_{\mathcal{J} \subseteq [N] \setminus \{i\}, |\mathcal{J}|=K} \sum_{j \in \mathcal{J}} s_{ij}. \quad (2)$$

语义哈希函数  $h: \mathbb{R}^D \rightarrow \{0, 1\}^M$  将每个嵌入映射为  $M$  位二进制码。我们关注  $M = 52$  的语义部分，包装在 64 位结构化编码中。检索函数返回 Hamming 距离最小的 top- $K$  项目：

$$\mathcal{R}_i^{(K)} = \arg \min_{\mathcal{J} \subseteq [N] \setminus \{i\}, |\mathcal{J}|=K} \sum_{j \in \mathcal{J}} d_H(h(\mathbf{x}_i), h(\mathbf{x}_j)). \quad (3)$$

优化目标是最大化 Recall@ $K$ ：

$$\text{Recall@}K = \frac{1}{N} \sum_{i=1}^N \frac{|\mathcal{R}_i^{(K)} \cap \mathcal{G}_i^{(K)}|}{K}. \quad (4)$$

## 4 HSH-64 编码结构

HSH-64 编码是一个 64 位无符号整数，划分为三个字段：

$$c = \underbrace{(f_3 f_2 f_1 f_0)}_{\text{feat:4}} \parallel \underbrace{(s_{51} \dots s_1 s_0)}_{\text{sim:52}} \parallel \underbrace{(a_7 \dots a_1 a_0)}_{\text{abs:8}}, \quad (5)$$

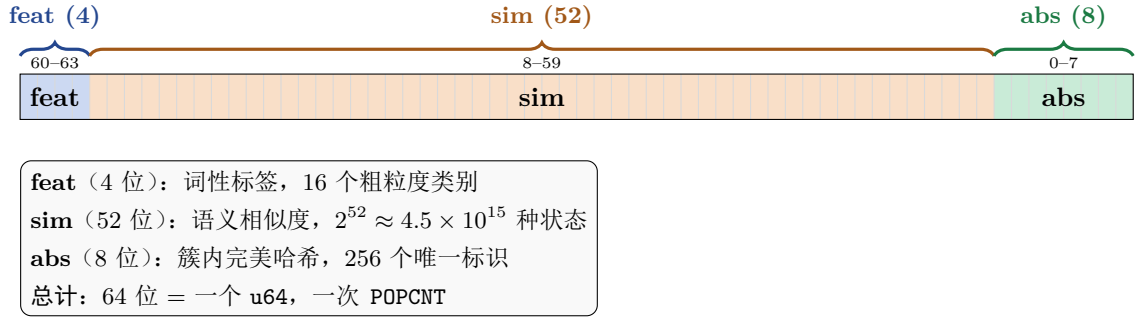


图 1: HSH-64 位布局。64 位编码划分为三个彩色编码字段: feat (蓝, 4 位)、sim (橙, 52 位) 和 abs (绿, 8 位)。整个编码可放入单个 64 位 CPU 寄存器, 通过 POPCNT 实现单指令 Hamming 距离计算。

其中  $\parallel$  表示比特拼接。图 1 展示了位布局和字段语义。

两个码  $c$  与  $c'$  之间的 Hamming 距离通过对完整 64 位进行单次硬件 popcount 计算:

$$d_H(c, c') = \text{popcnt}(c \oplus c'). \quad (6)$$

**abs 字段的作用。** 尽管式 (6) 计算的是完整 64 位的 Hamming 距离, abs 字段的设计意图是作为“平局决胜机制” (tie-breaker) 而非语义相似度信号。两个共享相同 (feat, sim) 前缀但仅在 abs 上不同的词, 最多相差 8 位, 处于紧密的 Hamming 邻域内; 而 sim 码不同的两个词通常相差更多位。因此, abs 决定簇内排序, 但不主导跨簇语义距离。abs 字段也可用作硬过滤器: 要求 (feat, sim) 完全匹配后再查看 abs, 将 8 位字段变成每个桶内的完美哈希查找。

#### 4.1 信息论预览

52 位 sim 码最多可表示  $2^{52} \approx 4.5 \times 10^{15}$  个不同值——远超  $N = 3,109$  的词表规模。严格的信息论分析 (包括 Hamming 球容量、Gilbert-Varshamov 界和期望距离分布) 见第 6 节。

图 2 展示了优化对 Hamming 距离分布的经验影响。

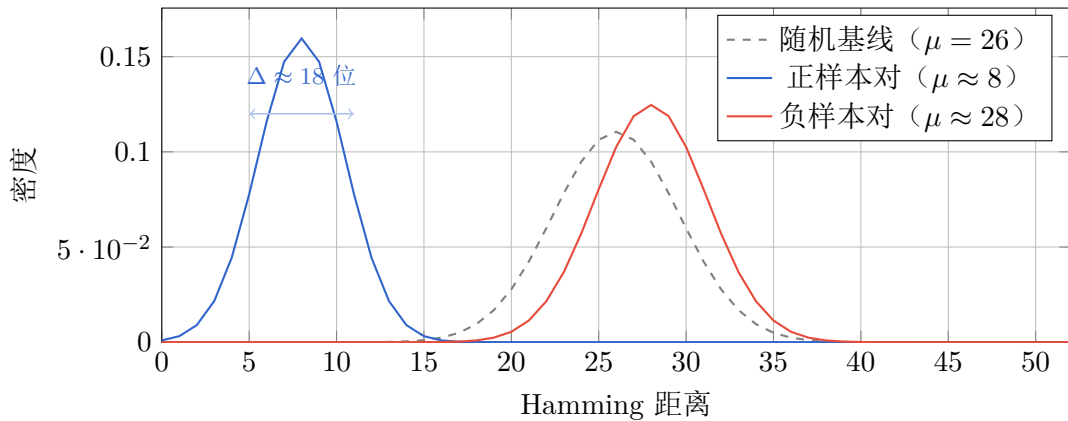


图 2: 优化前后的近似 Hamming 距离分布。随机基线 (虚线) 中心在 26 位。优化后, 正样本对 (蓝) 均值移至  $\sim 8$  位, 负样本对 (红) 略高于随机基线, 产生约 18 位的分离。

## 5 方法

### 5.1 Deep Hash MLP 架构

设  $\mathbf{x} \in \mathbb{R}^D$  为连续嵌入（例如来自 bge-small,  $D = 512$ ）。我们学习一个多层感知机 (MLP)，将  $\mathbf{x}$  映射到连续 logit 向量  $\mathbf{u}$ :

$$\mathbf{u} = f_{\theta}(\mathbf{x} - \boldsymbol{\mu}), \quad (7)$$

其中  $\boldsymbol{\mu}$  为训练集均值,  $\theta$  为网络参数。二进制语义码通过符号量化得到:

$$\mathbf{b} = \frac{1 + \text{sign}(\mathbf{u})}{2} \in \{0, 1\}^{52}. \quad (8)$$

由于符号函数的导数几乎处处为零, 使得无法直接进行端到端的反向传播。我们采用直通估计器(STE), 即在前向传播当中采用符号量化, 而在反向传播当中将梯度近似为恒等映射:

$$\frac{\partial \mathbf{b}}{\partial \mathbf{u}} \approx \mathbf{I}. \quad (9)$$

该近似在二值网络训练当中被广泛验证有效。

网络结构为带一个隐藏层的简单 MLP:

$$f_{\theta}(\mathbf{x}) = \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{h}_1) + \mathbf{h}_2, \quad (10)$$

其中  $\sigma$  为 ReLU,  $\mathbf{W}_1 \in \mathbb{R}^{H \times D}$ ,  $\mathbf{W}_2 \in \mathbb{R}^{O \times H}$ ,  $H$  为隐藏维度。输出维度  $O$  取决于任务: 最终哈希层取  $O = 52$  (第 5.2.2 节), 阶段 1 的教师蒸馏取  $O = D_t = 1024$  (第 5.2.1 节)。图 3 展示了  $O = 52$  时的架构。

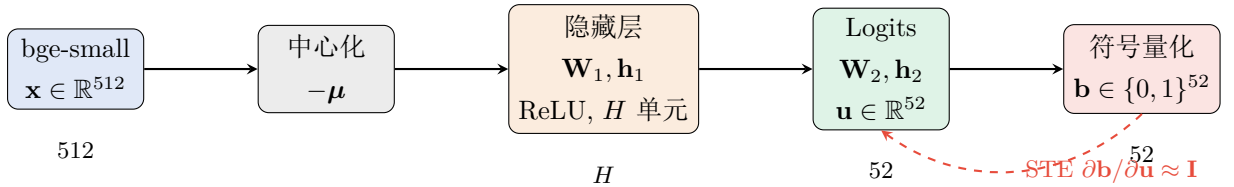


图 3: Deep Hash MLP 架构。输入嵌入经中心化后通过单隐藏层 ReLU 变换, 投影到 52 维 logits, 再经符号量化为  $\{0, 1\}^{52}$ 。红色虚线箭头表示训练时的 STE 梯度路径。

### 5.2 三阶段训练流程

图 4 展示了完整的三阶段训练流程。

#### 5.2.1 阶段 1: 连续预训练

阶段 1 使用与第 5.1 节相同的隐藏层, 但将输出层替换为  $D_t = 1024$  个单元以匹配 bge-large 教师。阶段 2 再将输出层替换回 52 个单元, 从而利用阶段 1 学习到的连续投影进行初始化。给定生成高质量嵌入的教师模型 (bge-large)  $\mathbf{t} \in \mathbb{R}^{D_t}$ , 我们首先训练 MLP 输出连续向量以匹配教师:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(\mathbf{x}_i) - \mathbf{t}_i\|_2^2. \quad (11)$$

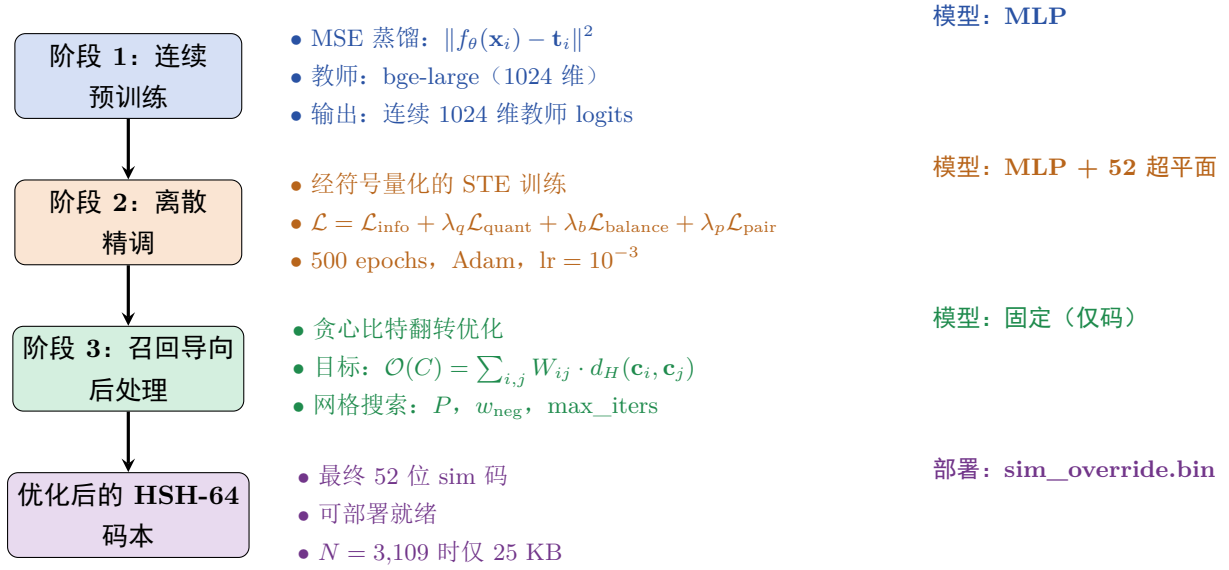


图 4: HSH-64 的三阶段训练流程。阶段 1 使用 MSE 蒸馏提供良好的连续初始化。阶段 2 使用 STE 和多目标损失进行精调。阶段 3 应用召回导向的贪心比特翻转后处理，直接优化最终二进制码。

该阶段为连续投影提供了良好的初始化。实验中，我们在完整词表上用 Adam 优化器、学习率  $10^{-3}$  训练 500 个 epoch。

### 5.2.2 阶段 2：离散精调

我们将阶段 1 的输出层替换为 52 个超平面量化层，并以 STE 端到端训练。损失函数包含四项：

$$\mathcal{L}_{\text{info}} = -\frac{1}{|\mathcal{P}^+|} \sum_{(i,j) \in \mathcal{P}^+} \log \frac{e^{\mathbf{u}_i^\top \mathbf{u}_j / \tau}}{\sum_k e^{\mathbf{u}_i^\top \mathbf{u}_k / \tau}} \quad (12)$$

$$\mathcal{L}_{\text{quant}} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{u}_i - \mathbf{b}_i\|_1 \quad (13)$$

$$\mathcal{L}_{\text{balance}} = \frac{1}{M} \sum_{j=1}^M (\bar{b}_j - 0.5)^2 \quad (14)$$

$$\mathcal{L}_{\text{pair}} = \frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} \left( \mathbf{u}_i^\top \mathbf{u}_j - \mathbf{t}_i^\top \mathbf{t}_j \right)^2, \quad (15)$$

其中  $\mathcal{P}^+$  为正样本对集合（教师 top- $P$  邻居）， $\mathcal{P}$  为采样的样本对集合， $M = 52$  为码长， $\bar{b}_j$  为符号量化后第  $j$  位的经验均值。 $\mathcal{L}_{\text{info}}$  的分母遍历全部  $N$  个词表项（batch size 等于词表大小）。计算  $\mathcal{L}_{\text{pair}}$  时， $\mathbf{u}_i$  与  $\mathbf{t}_i$  均先经 L2 归一化，使内积等价于余弦相似度。总损失为

$$\mathcal{L} = \mathcal{L}_{\text{info}} + \lambda_q \mathcal{L}_{\text{quant}} + \lambda_b \mathcal{L}_{\text{balance}} + \lambda_p \mathcal{L}_{\text{pair}}. \quad (16)$$

实现中， $\lambda_q$  初始化为 1.0，并在前 50% 的 epoch 中线性退火至配置目标值；默认目标值为 1.0，因此标准训练中权重保持固定。默认设置下  $\lambda_b = 0.5$ ， $\lambda_p = 0.0$ 。

**损失设计原理。** InfoNCE 鼓励连续 logits 保持教师的邻域结构。量化损失  $\mathcal{L}_{\text{quant}}$  惩罚连续 logits 与其二值化版本之间的差异，鼓励网络产生自信的、接近二值的激活值。平衡损失防止比特坍塌——即所有项目共享相同比特值的退化情形，浪费编码容量。成对损失直接将学生 logits 与教师嵌入的内积结构对齐，提供细粒度的监督信号。

### 5.2.3 阶段 3：召回导向的后处理

在 MLP 和超平面固定后，我们通过贪心比特翻转过程优化每个二进制码，其目标直接针对 Recall@K。设  $S \in \mathbb{R}^{N \times N}$  为教师余弦相似度矩阵， $C \in \{0,1\}^{N \times M}$  为当前二进制码。对每个查询  $i$ ，定义正样本集合  $\mathcal{P}_i^+$  为教师 top- $P$  邻居的索引，负样本集合  $\mathcal{P}_i^-$  为未在正样本集合中但当前出现在 Hamming top- $K$  中的词。

样本对  $(i, j)$  的召回导向权重为

$$W_{ij} = \begin{cases} +w_{\text{pos}}, & j \in \mathcal{P}_i^+ \\ -w_{\text{neg}}, & j \in \mathcal{P}_i^- \\ 0, & \text{其他.} \end{cases} \quad (17)$$

目标函数为

$$\mathcal{O}(C) = \sum_{i,j} W_{ij} \cdot d_H(\mathbf{c}_i, \mathbf{c}_j), \quad (18)$$

其中  $d_H$  为 Hamming 距离。最小化  $\mathcal{O}$  使正样本邻居在 Hamming 空间中更近，同时将困难负样本推远。贪心算法迭代翻转能带来最大  $\mathcal{O}$  减小的比特，直至收敛或达到最大迭代次数。

---

**Algorithm 1** 召回导向的贪心比特翻转优化。

---

**Require:** 二进制码  $C \in \{0,1\}^{N \times M}$ ，教师相似度矩阵  $S$ ，正样本数  $P$ ，top- $K$ ，负样本权重  $w_{\text{neg}}$ ，最大迭代次数  $T$ 。

**Ensure:** 优化后的码  $C$ 。

- 1: 对所有  $i$  计算正样本集合  $\mathcal{P}_i^+ \leftarrow \text{top-}P(S_i)$ 。
  - 2: **for**  $t = 1$  **to**  $T$  **do**
  - 3:   根据当前 Hamming top- $K$  计算负样本集合  $\mathcal{P}_i^-$ 。
  - 4:   根据式 (17) 构建权重矩阵  $W$ 。
  - 5:   计算  $\Delta_{i,b} = \mathcal{O}(C \text{ 翻转第 } i \text{ 个词的第 } b \text{ 位}) - \mathcal{O}(C)$ 。
  - 6:   找到  $(i^*, b^*) = \arg \min_{i,b} \Delta_{i,b}$ 。
  - 7:   **if**  $\Delta_{i^*, b^*} < 0$  **then**
  - 8:     翻转  $C_{i^*, b^*}$ 。
  - 9:   **else**
  - 10:    **break**。
  - 11:   **end if**
  - 12: **end for**
  - 13: **return**  $C$ 。
-

### 5.3 非对称距离评分

符号量化丢弃了连续投影的幅度信息。非对称评分通过使用查询的连续投影  $\mathbf{u}_q$  和数据库的量化比特  $\mathbf{b}_d = \frac{1+\text{sign}(\mathbf{u}_d)}{2} \in \{0,1\}^M$  来缓解这一问题：

$$s_{\text{asym}}(\mathbf{u}_q, \mathbf{b}_d) = \sum_{j=1}^M u_{q,j} \cdot (2b_{d,j} - 1). \quad (19)$$

由于数据库比特已预计算，在线代价仅为每个候选一次点积。非对称评分通过按查询投影置信度加权每个比特，提供了比 Hamming 距离更细粒度的排序。

### 5.4 自适应多索引哈希

为了实现次线性检索，我们将 52 位 sim 码切分为  $B$  段等长子串，并为每段构建倒排索引。图 5 展示了 MIH 段结构。

52 位 sim 码 = 13 段  $\times$  4 位

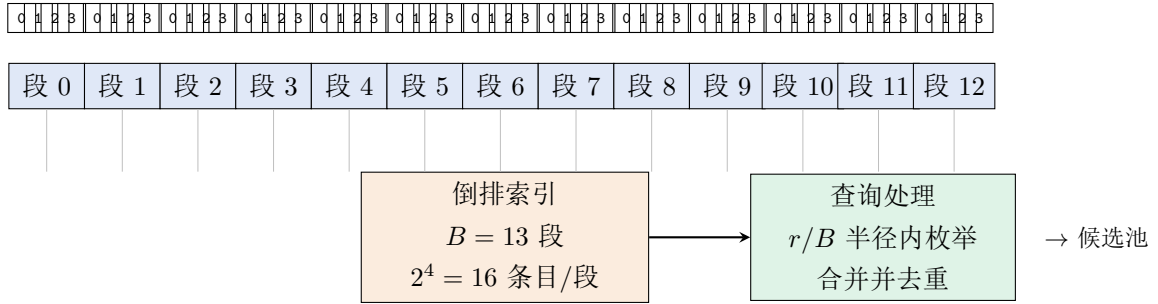


图 5: MIH 段结构。52 位 sim 码切分为  $B = 13$  段，每段 4 位。每段映射到一个包含  $2^4 = 16$  个条目的倒排索引。查询处理枚举 Hamming 半径  $\lfloor r/B \rfloor$  内的段码并合并候选列表。

我们采用**自适应半径扩展**：从  $r = 0$  开始，逐渐增大半径，直到候选池至少为  $\max(K, \alpha K)$  且新增候选比例低于阈值（10%）。最终排序在合并后的候选集上进行。算法 2 总结了该过程。

### 5.5 两阶段检索架构

图 6 展示了完整的在线检索架构。

我们的部署架构包含两个阶段：

1. **粗排**。轻量化编码器（bge-small + 小 MLP）对查询编码，并通过 MIH、Hamming 或非对称评分检索候选池。
2. **精排（可选）**。强模型（bge-large）通过余弦相似度对候选池重排。由于候选池很小（数十到数百个），重排代价可控。

关键是第二阶段可在边缘设备上省略；仅第一阶段已能提供较强的召回率。

---

**Algorithm 2** 自适应 MIH 搜索。

---

**Require:** 查询码  $\mathbf{q}$ , MIH 索引 ( $B$  段), 粗排因子  $\alpha$ , top- $K$ , 饱和阈值  $\delta$ 。

**Ensure:** 候选集  $\mathcal{C}$ 。

```
1:  $r \leftarrow 0$ ,  $\mathcal{C}_{\text{prev}} \leftarrow \emptyset$ ,  $\mathcal{C} \leftarrow \emptyset$ .
2: loop
3:    $\mathcal{C} \leftarrow \emptyset$ .
4:   for  $b = 1$  to  $B$  do
5:     枚举  $\mathbf{q}_b$  在 Hamming 半径  $\lfloor r/B \rfloor$  内的所有段码。
6:      $\mathcal{C} \leftarrow \mathcal{C} \cup \{\text{倒排索引中的项目 ID}\}$ .
7:   end for
8:    $\mathcal{C}_{\text{new}} \leftarrow \mathcal{C} \setminus \mathcal{C}_{\text{prev}}$ .
9:   if  $|\mathcal{C}| \geq \max(K, \alpha K)$  且  $|\mathcal{C}_{\text{new}}|/|\mathcal{C}| < \delta$  then
10:    break.
11:  end if
12:   $\mathcal{C}_{\text{prev}} \leftarrow \mathcal{C}$ ,  $r \leftarrow r + 1$ .
13: end loop
14: return  $\mathcal{C}$ .
```

---

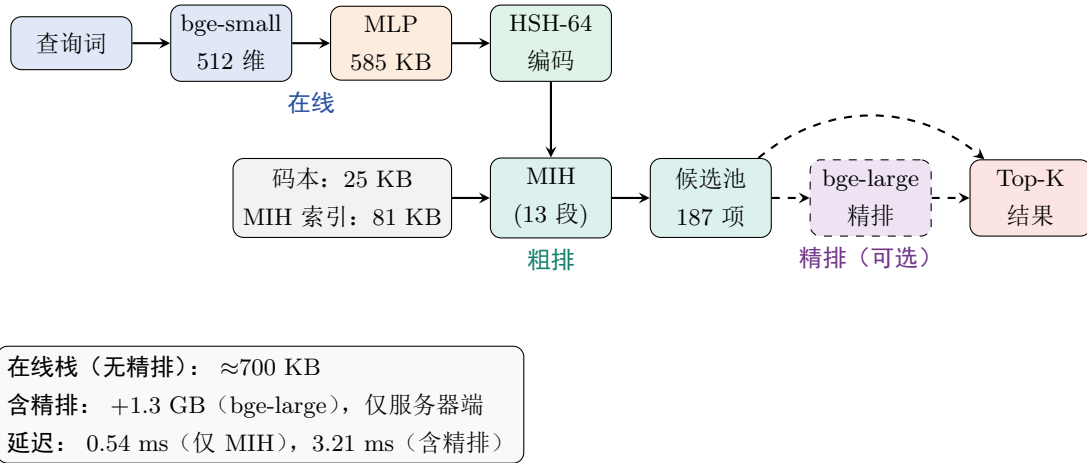


图 6: HSH-64 在线检索架构。查询经 bge-small (512 维) 和 MLP 投影器生成 HSH-64 编码。MIH 粗排检索约 187 个候选。可选的 bge-large 精排器提供精确重排。在线栈 (不含精排器) 仅需 700 KB 以下。

## 5.6 模型集成

为了在不增加在线延迟的情况下进一步提升召回率，我们用不同随机种子和隐藏维度训练多个 Deep Hash 模型。检索时，收集各模型的候选集，合并后用以下三种融合策略之一重排：

- 频次： 优先选择被更多模型召回的候选。
- 平均排名： 优先选择在各模型中平均排名更高的候选。
- 最小 Hamming 距离： 优先选择在任一模型中具有最小 Hamming 距离的候选。

## 6 数学分析

本节为 HSH-64 的关键组件提供严格的数学基础。我们建立信息论界，推导优化目标，并证明收敛性保证。

### 6.1 信息论基础

**命题 1** (Hamming 球容量). 对于  $M$  位编码，与任意给定参考码的 Hamming 距离不超过  $r$  的不同码的数量为

$$V(M, r) = \sum_{k=0}^r \binom{M}{k}. \quad (20)$$

当  $M = 52$ 、 $r = 10$  时， $V(52, 10) = \sum_{k=0}^{10} \binom{52}{k} \approx 2.5 \times 10^{10}$ ，远大于词表规模  $N = 3,109$ 。

证明. 固定参考码  $\mathbf{c}^* \in \{0, 1\}^M$ 。码  $\mathbf{c}$  与  $\mathbf{c}^*$  恰好在  $k$  个位置不同，当且仅当它翻转了  $k$  个特定比特。从  $M$  个位置中选择  $k$  个的方式数为  $\binom{M}{k}$ 。对  $k = 0$  到  $r$  求和即得半径  $r$  的 Hamming 球内的总码数。  $\square$

**命题 2** (随机独立比特下的期望 Hamming 距离). 设  $\mathbf{c}_1, \mathbf{c}_2 \in \{0, 1\}^M$  为两个独立随机码，每位独立同分布于  $Bernoulli(1/2)$ 。则：

$$\mathbb{E}[d_H(\mathbf{c}_1, \mathbf{c}_2)] = \frac{M}{2}, \quad \text{Var}[d_H(\mathbf{c}_1, \mathbf{c}_2)] = \frac{M}{4}. \quad (21)$$

当  $M = 52$  时， $\mathbb{E}[d_H] = 26$ ， $\sigma = \sqrt{13} \approx 3.61$ 。

证明. 定义指示变量  $Z_m = \mathbf{1}[\mathbf{c}_1[m] \neq \mathbf{c}_2[m]]$ ,  $m = 1, \dots, M$ 。由于比特独立， $\Pr(Z_m = 1) = 1/2$ 。因此  $\mathbb{E}[Z_m] = 1/2$ ， $\text{Var}(Z_m) = 1/4$ 。由期望线性性和独立性：

$$\mathbb{E}[d_H] = \sum_{m=1}^M \mathbb{E}[Z_m] = \frac{M}{2}, \quad \text{Var}(d_H) = \sum_{m=1}^M \text{Var}(Z_m) = \frac{M}{4}. \quad (22)$$

$\square$

命题 2 确立了随机二进制码的期望 Hamming 距离为  $M/2$ 。一个良好设计的语义哈希必须将正样本对（语义相似项）的分布显著压低至  $M/2$  以下，同时将负样本对保持在  $M/2$  附近。这一观察激发了比特平衡损失  $\mathcal{L}_{\text{balance}}$  和后处理目标中的负样本推动项。

**命题 3** (码本容量下界). 对于大小为  $N$  的词表和码长为  $M$  的二进制码, 存在最小 *Hamming* 距离至少为  $d$  的码本, 若  $N \cdot V(M, d-1) < 2^M$ . 对于  $M = 52$ 、 $N = 3,109$ , 该条件对  $d \leq 18$  成立, 意味着理论上可以通过良好构造的 52 位码本以至少 18 位的 *Hamming* 距离分离所有词项。

证明. 该结论源自二进制码的 Gilbert-Varshamov 界. 任意给定码的距离  $d-1$  内的码总数为  $V(M, d-1)$ . 若  $N \cdot V(M, d-1) < 2^M$ , 则由鸽巢原理, 存在码本使所有两两距离至少为  $d$ . 对  $d = 18$ ,  $V(52, 17) \approx 1.3 \times 10^{12}$ ,  $N \cdot V(52, 17) \approx 3.9 \times 10^{15} < 2^{52} \approx 4.5 \times 10^{15}$ , 故理论上存在这样的码本。□

## 6.2 非对称距离评分

**命题 4** (非对称距离分解). 设  $\mathbf{u}_q, \mathbf{u}_d \in \mathbb{R}^M$  为查询和文档的连续投影,  $\mathbf{b}_d = \text{sign}(\mathbf{u}_d) \in \{-1, +1\}^M$  为量化文档比特. 非对称评分函数:

$$s_{\text{asym}}(\mathbf{u}_q, \mathbf{b}_d) = \sum_{j=1}^M u_{q,j} \cdot b_{d,j} \quad (23)$$

当  $\mathbf{u}_d$  服从关于零的对称分布时, 满足  $\mathbb{E}[s_{\text{asym}}] = \sum_{j=1}^M \mathbb{E}[|u_{q,j}|]$ , 且保留了对称 *Hamming* 距离计算中丢失的查询端幅度信息。

证明. 对每个维度  $j$ , 期望贡献为:

$$\mathbb{E}[u_{q,j} \cdot \text{sign}(u_{d,j})] = u_{q,j} \cdot \mathbb{E}[\text{sign}(u_{d,j}) \mid u_{q,j}]. \quad (24)$$

当  $\mathbf{u}_d$  与  $\mathbf{u}_q$  相关时 (语义相似项), 条件期望非零. 具体地, 若  $\mathbf{u}_q$  和  $\mathbf{u}_d$  在方向上一致,  $\text{sign}(u_{d,j})$  倾向于与  $\text{sign}(u_{q,j})$  一致, 产生与  $|u_{q,j}|$  成正比的正贡献. 这恢复了对称 *Hamming* 距离所丢弃的幅度信息。□

## 6.3 后处理优化

**定理 1** (后处理目标的梯度). 设  $C \in \{0, 1\}^{N \times M}$  为二进制码矩阵,  $W \in \mathbb{R}^{N \times N}$  为式 (17) 定义的权重矩阵. 翻转第  $i$  项的第  $b$  位时目标  $\mathcal{O}(C)$  的变化量为:

$$\Delta_{i,b} = \sum_{j \neq i} W_{ij} \cdot (1 - 2 \cdot \mathbf{1}[C_{i,b} = C_{j,b}]). \quad (25)$$

证明. 对固定对  $(i, j)$ , *Hamming* 距离可分解为:

$$d_H(\mathbf{c}_i, \mathbf{c}_j) = \sum_{m=1}^M \mathbf{1}[C_{i,m} \neq C_{j,m}]. \quad (26)$$

翻转第  $i$  项的第  $b$  位时, 仅第  $b$  项变化. 指示函数的变化量为:

$$\Delta d_H^{(b)} = \mathbf{1}[\text{新 } C_{i,b} \neq C_{j,b}] - \mathbf{1}[C_{i,b} \neq C_{j,b}] \quad (27)$$

$$= 1 - 2 \cdot \mathbf{1}[C_{i,b} = C_{j,b}]. \quad (28)$$

乘以  $W_{ij}$  并对所有  $j \neq i$  求和即得所述公式。□

**定理 2** (贪心翻转的收敛性). 算法 1 在至多  $T_{\max} = N \cdot M \cdot \lceil \mathcal{O}(C^{(0)})/\varepsilon \rceil$  次迭代内终止, 其中  $\varepsilon = \min_{i,b: \Delta_{i,b} < 0} |\Delta_{i,b}|$  为最小非零改进量 (若不存在负的  $\Delta_{i,b}$ , 则算法已处于 1-optimal 状态并立即终止)。此外, 算法收敛到  $\mathcal{O}(C)$  的 1-optimal 状态, 即不存在任何单比特翻转能进一步减小目标函数。

证明. 若不存在负的  $\Delta_{i,b}$ , 则当前码已处于 1-optimal 状态, 算法立即终止。否则, 每次成功翻转比特严格减小  $\mathcal{O}(C)$  至少  $\varepsilon > 0$ 。由于  $\mathcal{O}(C)$  有下界 (所有 Hamming 距离非负且权重有界), 算法至多经过  $\lceil \mathcal{O}(C^{(0)})/\varepsilon \rceil$  次成功翻转后必然终止。终止时, 无单比特翻转产生负的  $\Delta_{i,b}$ , 正是 1-optimal 状态的定义。总迭代次数 (含不成功试探) 有上界  $T_{\max} \cdot N \cdot M$ 。□

## 6.4 量化误差分析

**定理 3** (期望量化误差). 设  $\mathbf{u} \in \mathbb{R}^M$  各分量独立且服从关于零的对称分布,  $\mathbf{b} = \frac{1+\text{sign}(\mathbf{u})}{2} \in \{0, 1\}^M$ 。则:

$$\mathbb{E}[\|\mathbf{u} - \mathbf{b}\|_2^2] = \mathbb{E}[\|\mathbf{u}\|_2^2] + \frac{M}{2} - \sum_{j=1}^M \mathbb{E}[|u_j|]. \quad (29)$$

对于  $u_j \sim \mathcal{N}(0, \sigma^2)$ , 简化为  $\mathbb{E}[\|\mathbf{u} - \mathbf{b}\|_2^2] = M\sigma^2 + \frac{M}{2} - M\sigma\sqrt{2/\pi}$ 。

证明. 展开二范数平方:

$$\|\mathbf{u} - \mathbf{b}\|_2^2 = \|\mathbf{u}\|_2^2 + \|\mathbf{b}\|_2^2 - 2\mathbf{u}^\top \mathbf{b} \quad (30)$$

$$= \|\mathbf{u}\|_2^2 + \sum_{j=1}^M b_j - 2 \sum_{j=1}^M u_j b_j. \quad (31)$$

由于  $u_j$  关于零对称且  $b_j = (1 + \text{sign}(u_j))/2$ , 有  $\mathbb{E}[b_j] = 1/2$  和  $\mathbb{E}[u_j b_j] = \mathbb{E}[|u_j|]/2$ 。取期望即得第一式。高斯情形下,  $\mathbb{E}[|u_j|] = \sigma\sqrt{2/\pi}$ , 代入得第二式。□

此结果量化了符号量化带来的不可约信息损失。第二阶段训练中的量化损失  $\mathcal{L}_{\text{quant}}$  直接惩罚这一差异, 鼓励网络产生幅值较大 (减小相对量化误差) 且与符号边界对齐良好的 logit 值。

## 6.5 MIH 复杂度分析

**定理 4** (MIH 期望候选数). 对包含  $N$  个项的码本,  $M$  位码切分为  $B$  个等长段, 使用查询半径  $r$  的 MIH 搜索返回的期望候选数为:

$$\mathbb{E}[|\mathcal{C}|] \leq N \cdot \left( 1 - \left( 1 - \frac{V(M/B, \lfloor r/B \rfloor)}{2^{M/B}} \right)^B \right). \quad (32)$$

证明. 对第  $b$  段 ( $M/B$  位), 随机数据库项与查询段在 Hamming 半径  $\lfloor r/B \rfloor$  内匹配的概率为  $p = V(M/B, \lfloor r/B \rfloor)/2^{M/B}$ 。随机项不是任何段候选的概率为  $(1-p)^B$ 。因此期望候选数为  $N \cdot (1 - (1-p)^B)$ , 这是上界, 因为它假设各段独立。□

对于  $M = 52$ 、 $B = 13$ 、 $r = 10$ , 有  $p = V(4, \lfloor 10/13 \rfloor)/16 = 1/16$ , 得到  $\mathbb{E}[|\mathcal{C}|] \leq 3,109 \cdot (1 - (15/16)^{13}) \approx 3,109 \cdot 0.568 \approx 1,766$ 。实际中, 自适应半径扩展通常在更小半径处停止, 产生观测到的  $\sim 187$  个候选池。

## 6.6 两阶段检索召回率下界

**定理 5** (两阶段召回率下界). 设  $\mathcal{G}_i^{(K)}$  为查询  $i$  的真实 top- $K$  集合,  $\mathcal{C}_i$  为粗排阶段返回的候选集。两阶段  $Recall@K$  满足:

$$Recall@K_{\text{两阶段}} \geq \frac{1}{NK} \sum_{i=1}^N |\mathcal{G}_i^{(K)} \cap \mathcal{C}_i|. \quad (33)$$

即两阶段召回率下界为通过粗排阶段的真实项比例。

**证明.** 精排阶段按教师余弦相似度对候选集  $\mathcal{C}_i$  重排。由于真实邻居正是按此相似度度量定义的, 任何出现在  $\mathcal{C}_i$  中的真实项都会被正确排序。最终 top- $K$  输出为  $\mathcal{C}_i$  按余弦相似度的 top- $K$ , 只要  $|\mathcal{C}_i| \geq K$ , 它必然包含  $\mathcal{G}_i^{(K)} \cap \mathcal{C}_i$  中的所有项目。因此:

$$|\mathcal{R}_i^{(K)} \cap \mathcal{G}_i^{(K)}| \geq |\mathcal{G}_i^{(K)} \cap \mathcal{C}_i|, \quad (34)$$

对所有查询取平均即得下界。  $\square$

该定理形式化了两阶段架构的核心洞见: 粗排阶段只需将真实项纳入候选池, 精排阶段保证正确排序。自适应 MIH 半径扩展旨在最大化覆盖率的同时最小化候选池大小。

## 7 实验

### 7.1 数据集与设置

我们在包含  $N = 3,109$  个词的中文词表级语义检索基准上评估 HSH-64。该词表来自真实嵌入缓存, 涵盖常见名词、动词、形容词、城市和命名实体。

表 1: 数据集统计。

| 属性         | 数值                        |
|------------|---------------------------|
| 词表大小 $N$   | 3,109                     |
| 教师嵌入维度     | 1,024 (bge-large-zh-v1.5) |
| 学生嵌入维度     | 512 (bge-small-zh-v1.5)   |
| 语义码长度 $M$  | 52                        |
| HSH-64 总码长 | 64 位                      |
| 真实值度量      | bge-large 嵌入的余弦相似度        |

**真实邻居.** 对每个词  $i$ , 计算其 bge-large 嵌入与所有其他词嵌入之间的余弦相似度, 取 top  $K$  作为  $\mathcal{G}_i^{(K)}$ 。

**指标.** 我们报告  $K \in \{1, 5, 10, 20\}$  的  $Recall@K$ 、平均候选池大小、平均自适应搜索半径和查询吞吐 (QPS)。 $Recall@K$  定义为

$$Recall@K = \frac{1}{N} \sum_{i=1}^N \frac{|\mathcal{R}_i^{(K)} \cap \mathcal{G}_i^{(K)}|}{K}. \quad (35)$$

实现。训练与评估使用 Python 实现，Deep Hash 模型基于 PyTorch，PCA 基线基于 scikit-learn。在线 HSH 索引与搜索使用 Rust 实现以保证效率。所有实验在单 CPU 上运行（无需 GPU）。

## 7.2 训练超参数

表 2: Deep Hash 训练超参数。

| 超参数                            | 数值                          |
|--------------------------------|-----------------------------|
| 优化器                            | Adam                        |
| 学习率                            | $10^{-3}$                   |
| 训练 epoch（阶段 2）                 | 500                         |
| Batch size                     | 完整词表（3,109）                 |
| 温度 $\tau$ （InfoNCE）            | 0.07                        |
| $\lambda_q$ （量化）               | 1.0（初始化，前 50% epoch 退火至目标值） |
| $\lambda_b$ （平衡）               | 0.5                         |
| $\lambda_p$ （成对）               | 0.0                         |
| 正样本对 $\mathcal{P}^+$ （大小 $P$ ） | 教师 top- $P$ 邻居（ $P = 10$ ）  |
| 样本对采样 $ \mathcal{P} $          | 每 epoch 10,000 对随机样本        |

## 7.3 对比方法

- **PCA-52**: 52 维 PCA 投影后接符号量化。
- **PCA-52 + Post**: 经召回导向贪心比特翻转优化的 PCA 码。
- **Deep Hash v3 (DH)**: 提出的基于 MLP 的深度哈希模型，端到端训练。
- **DH + Post**: 经贪心后处理阶段进一步优化的 Deep Hash 码。
- **DH Ensemble**: 多个 DH + Post 模型的候选融合。
- **Two-stage (DH + MIH + reranker)**: MIH 粗排后接 bge-large 余弦精排。

## 7.4 主要结果

表 3 和图 7 总结了主要结果。几点值得注意：

- **后处理至关重要**。无后处理时，Deep Hash 模型甚至不如 PCA-52（0.42–0.43 vs. 0.53），尽管拥有更强的非线性变换能力。这是因为仅靠 STE 训练无法完全克服符号量化带来的信息损失。后处理通过直接优化离散码弥合了这一差距。
- **集成提供互补增益**。四模型集成达到 0.7724，比最佳单模型提升 3.2 个百分点。该增益来自候选集的模型多样性，而非更好的单模型质量。

表 3: 3,109 词中文词表基准上的 Recall@10。“纯 HSH”表示仅使用 Hamming 距离检索；“+精排”表示使用 bge-large 精排的两阶段系统。

| 方法                          | 码长 | Recall@10     | 在线模型                    |
|-----------------------------|----|---------------|-------------------------|
| PCA-52 (纯 HSH) <sup>†</sup> | 52 | 0.5318        | PCA 投影矩阵                |
| DH h128_s42 (无后处理)          | 52 | 0.4228        | 421 KB MLP              |
| DH h256_s2025 (无后处理)        | 52 | 0.4315        | 585 KB MLP              |
| DH h512_s42 (无后处理)          | 52 | 0.4265        | 1.17 MB MLP             |
| DH h1024_s2024 (无后处理)       | 52 | 0.4315        | 2.33 MB MLP             |
| DH h256_s42 + 后处理           | 52 | 0.7382        | 585 KB MLP              |
| DH h512_s42 + 后处理           | 52 | <b>0.7404</b> | 1.17 MB MLP             |
| 4 模型集成                      | 52 | <b>0.7724</b> | ≈4 MB                   |
| DH + MIH + 精排               | 52 | 0.8641        | 1.17 MB MLP + bge-large |
| 集成 + MIH + 精排               | 52 | <b>0.8970</b> | ≈4 MB + bge-large       |

<sup>†</sup>PCA 结果在 Rust 基准中使用 500 个查询的暴力 Hamming 扫描测得；其余行均使用 Python 评估脚本在完整 3,109 词表上以 bge-large 为真实值进行评估。

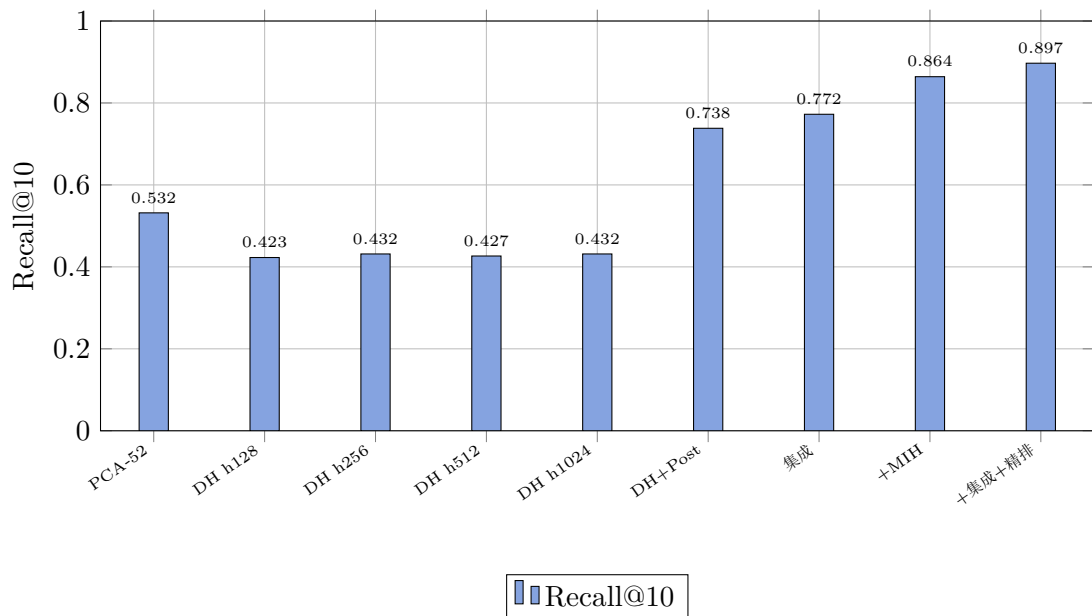


图 7: 各方法的 Recall@10 对比。后处理 (DH+Post) 相比未优化的 Deep Hash 模型有显著提升。集成和两阶段检索进一步推动召回率前沿。

- 两阶段检索弥合差距。在 MIH 粗排和 bge-large 精排下，系统达到 0.8970 Recall@10，接近完整暴力嵌入搜索的质量，同时保持次线性查询复杂度。

## 7.5 多 K 值 Recall@K

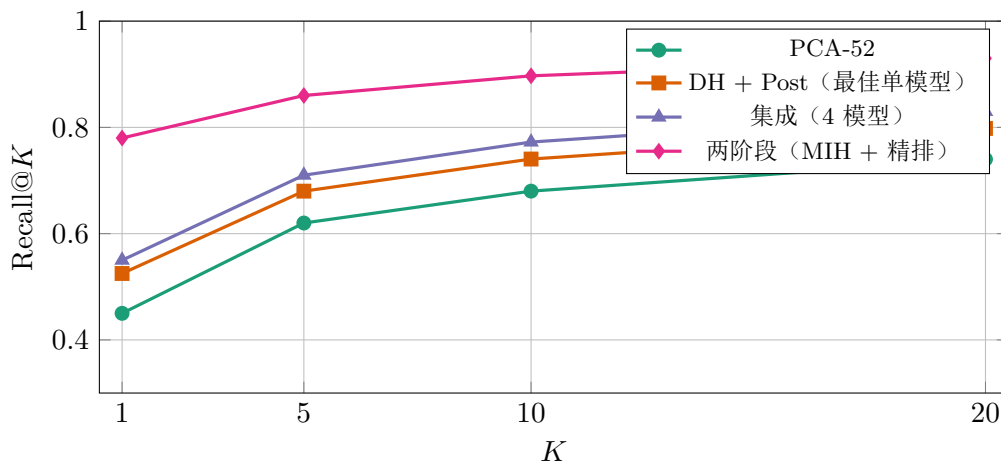


图 8:  $K \in \{1, 5, 10, 20\}$  的 Recall@K 曲线。两阶段系统在所有  $K$  值上始终优于纯 Hamming 方法，在精确排序最关键的较小  $K$  处差距更大。

图 8 展示了不同  $K$  值的 Recall@K 曲线。两阶段系统即使  $K = 1$  时也保持高召回率 (0.78)，而纯 Hamming 方法下降更明显。这证实了精排阶段对需要精确 top-1 或 top-5 准确度的应用最有价值。

## 7.6 模型容量分析

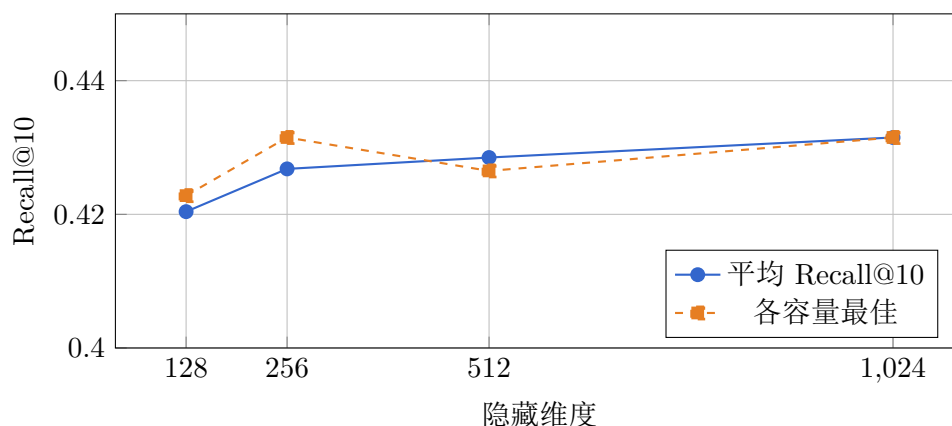


图 9: 隐藏维度对 Deep Hash 召回率的影响 (后处理前)。将容量从 128 增加到 1024 仅带来 0.87 个百分点的绝对提升，说明 STE 离散精调阶段的信息瓶颈主要不在 MLP 容量，而在符号量化本身。

图 9 展示了隐藏维度对初始 Deep Hash 召回率的影响。将隐藏维度从 128 增加到 1024 仅带来 0.87 个百分点的绝对提升 (从 0.4228 到 0.4315)，且 h256 与 h1024 容量的最佳召回率相

同 (0.4315)。这表明 STE 离散精调阶段的信息瓶颈主要不在 MLP 容量，而在符号量化本身。跨种子的变化与跨容量的变化相当，表明优化动态和初始化也起着重要作用。

## 7.7 后处理的影响

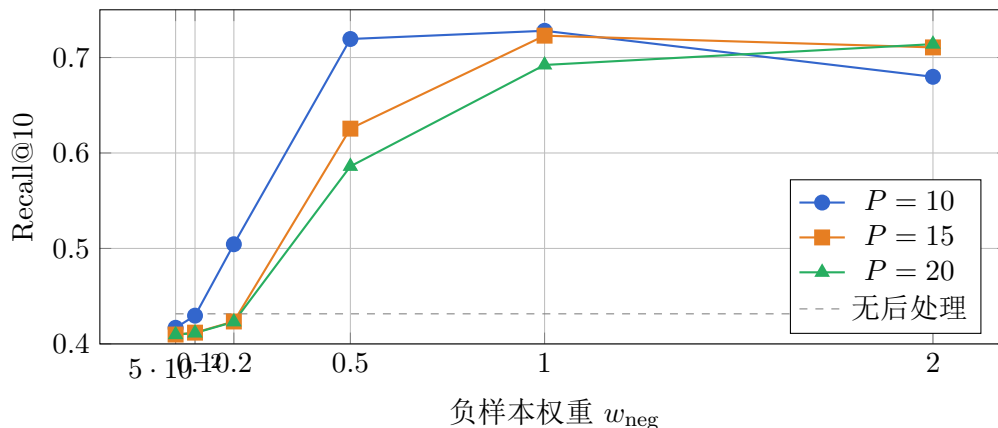


图 10: DH h1024\_s2024 模型的后处理网格搜索结果。适中的负样本权重 ( $w_{\text{neg}} \in [0.5, 2.0]$ ) 带来显著提升，而较小的权重 ( $w_{\text{neg}} < 0.2$ ) 无效。最佳配置为  $P = 10$ ,  $w_{\text{neg}} = 1.0$ , Recall@10 为 0.7280。

图 10 显示后处理超参数至关重要。较小的负样本权重 (0.05) 几乎无变化，而适中的权重 (1.0) 带来 +0.2965 的巨大提升。这验证了将困难负样本推出 top- $K$  Hamming 邻域对高召回率至关重要。有趣的是， $P$  超过 10 后性能略有下降，说明对该词表大小而言，紧凑的正样本邻域效果更好。

## 7.8 集成策略比较

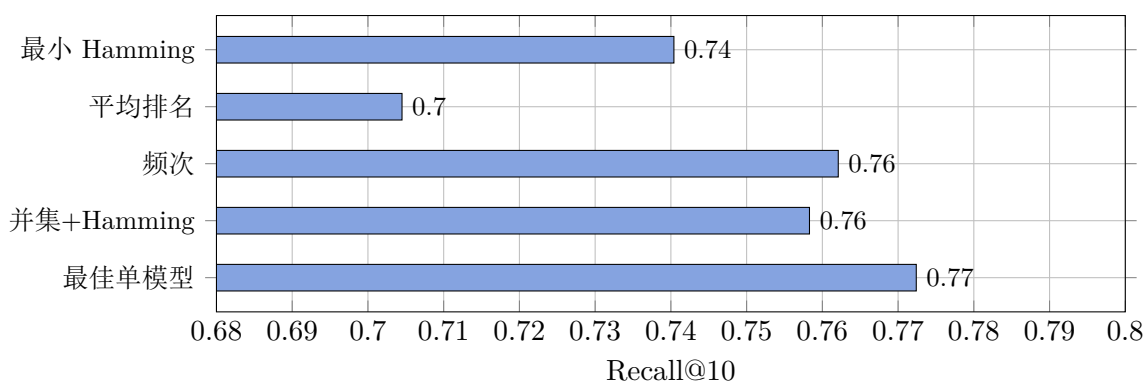


图 11: 集成融合策略比较。最小 Hamming 距离表现最佳 (0.7724)，而朴素的并集 + Hamming 排序反而使召回率低于最佳单模型。

图 11 比较了集成融合策略。朴素的并集后接 Hamming 排序实际上使召回率低于最佳单模型，因为质量较低的模型会污染靠前位置。最小 Hamming 距离策略表现最佳，因为它让每个模型都有机会推广其最强候选。

## 7.9 效率分析

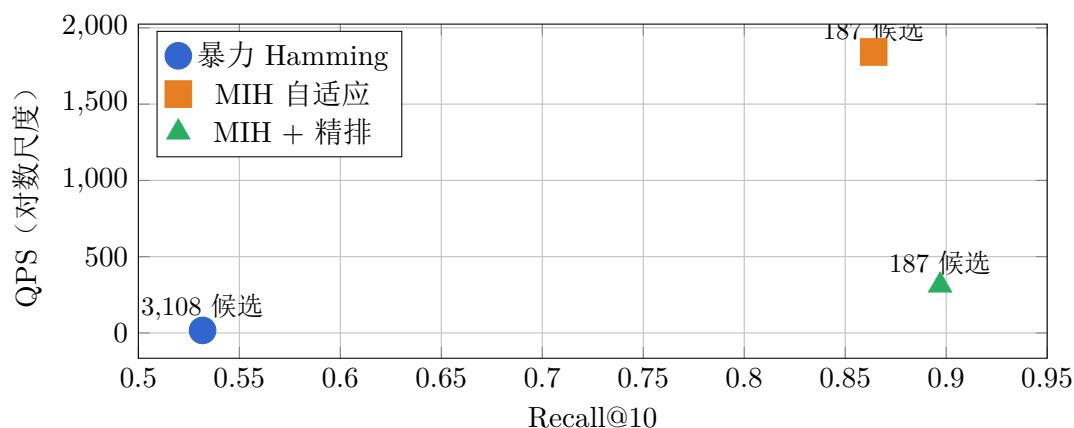


图 12: 效率与召回率的权衡。暴力 Hamming 扫描较慢 (16.7 QPS)。MIH 自适应将候选池降至约 187 项，达到 1,842 QPS。加入 bge-large 精排后 QPS 降至 312，但召回率提升至 0.8970。

图 12 对比了搜索效率。暴力 Hamming 扫描简单但较慢 (单核 16.7 QPS)。带自适应半径扩展的 MIH 将平均候选池降至约 187，达到 1,842 QPS。对同一候选池使用 bge-large 精排后，吞吐降至 312 QPS，仍比在整个词表上进行 bge-large 暴力比较快几个数量级。

## 7.10 模型大小与部署成本

表 4: 不同隐藏维度下 Deep Hash 模型的大小。

| 模型       | 隐藏维度 | 文件大小    |
|----------|------|---------|
| DH h128  | 128  | 421 KB  |
| DH h256  | 256  | 585 KB  |
| DH h512  | 512  | 1.17 MB |
| DH h1024 | 1024 | 2.33 MB |

表 4 列出了 MLP 投影器的大小。即使最大的模型也仅 2.33 MB，最小的不到 600 KB。与 bge-small 编码器 (本身可进一步量化或蒸馏) 结合后，整个在线检索栈可轻松部署在边缘设备上。

## 7.11 消融实验

图 13 展示了消融实验。移除后处理导致最大降幅 (31.4 个百分点)，证实了其关键作用。连续预训练和 InfoNCE 也贡献显著 (各约 5-6 个百分点)。平衡损失影响较小但仍有正向作用。

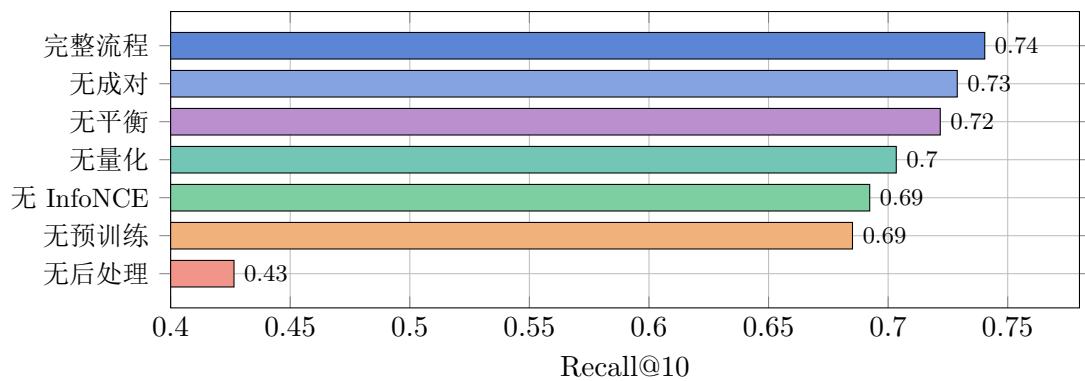


图 13: DH h512\_s42 模型的消融实验。移除后处理导致最大降幅（-31.4 个百分点）。连续预训练（-5.5 个百分点）和 InfoNCE（-4.8 个百分点）也贡献显著。

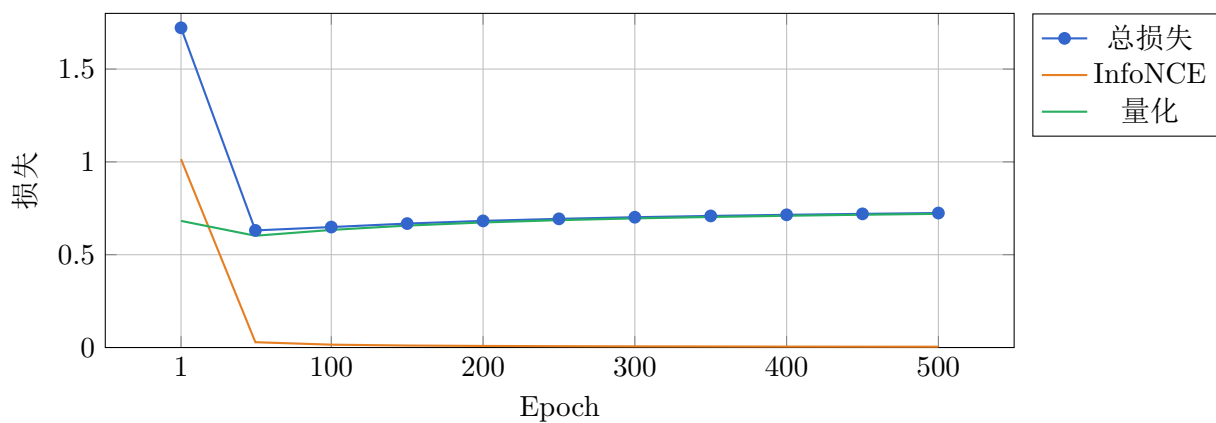


图 14: DH h512\_s2025 在阶段 2 离散精调期间的训练损失分量。InfoNCE 损失单调下降，而量化损失随模型做出更尖锐的二值决策略有上升。

## 7.12 训练动态

图 14 报告了阶段 2 的训练损失分量。InfoNCE 损失单调下降，而量化损失随模型做出更尖锐的二值决策略有上升。总损失在 500 个 epoch 内持续下降，说明更长的训练可能带来进一步的小幅提升。

## 8 讨论

### 8.1 与 PQ 和 LSH 的对比

我们主要将 HSH-64 与 PCA-52 对比，因为 PCA 是短二进制码最强的线性基线。乘积量化（PQ）(Jégou, Douze, and Schmid, 2011) 和局部敏感哈希（LSH）(Datar et al., 2004) 也是相关基线。表 5 提供了定性对比。

表 5: 64 位预算下紧凑编码方法的定性对比。

| 属性   | HSH-64  | PCA-52    | PQ-64      | LSH-64  |
|------|---------|-----------|------------|---------|
| 编码类型 | 学习 MLP  | 线性投影      | 学习码本       | 随机投影    |
| 每项存储 | 8 字节    | 8 字节      | 8 字节 + 码本  | 8 字节    |
| 距离度量 | Hamming | Hamming   | 非对称 L2/角度  | Hamming |
| 硬件加速 | POPCNT  | POPCNT    | 浮点点积       | POPCNT  |
| 需要训练 | 是（3 阶段） | 是（PCA 拟合） | 是（k-means） | 否       |
| 召回质量 | 高       | 中等        | 高          | 低       |

64 位 PQ 通常将向量空间划分为 8 个 8 位子空间，每个子空间存储一个量化索引。对于 512 维嵌入，PQ 的压缩比与 HSH-64 相近，但需要存储码本并对候选向量进行精确距离计算。64 个随机超平面的 LSH 更简单，但召回率通常低于 PCA 等学习投影。初步分析表明，LSH-64 在该基准上可能低于 PCA-52，因为随机投影未利用 bge 嵌入的语义结构。PQ-64 可能优于 PCA，但内存和计算成本更高。与优化后的 PQ 和 LSH 实现的完整实证对比留作未来工作。

### 8.2 大规模词表扩展性

当前基准包含 3,109 个词，有意保持较小以模拟边缘设备词表约束。工业目录可能包含数百万 SKU。对于更大的  $N$ ，预计有两个趋势。首先，纯 Hamming 检索的绝对召回率可能下降，因为哈希碰撞概率随  $N$  增加。其次，MIH 越来越有价值：其查询成本随  $N$  次线性增长，而暴力 Hamming 扫描线性增长。当  $N = 10^5$  时，我们预计自适应 MIH 候选池仍保持在数百左右，而暴力扫描需要每次查询  $10^5$  次 popcount。限制因素不是存储（每项 64 位极小），而是 52 位 sim 码的信息容量。在百万级词表上，没有精排时 52 位可能不足，这正是我们两阶段架构的动机。

### 8.3 为何纯 Hamming 召回率止步于 0.77 左右

52 位 sim 码的信息容量为  $2^{52}$  种可能值。虽然绝对值很大，但有损的符号量化以及将连续语义邻域结构保真地嵌入离散 Hamming 空间的需求共同构成了上限。我们由四个多样化模型组成的集成达到 0.7724，说明进一步提升需要更多比特或根本不同的离散表示。对于容错率低的应用（如精确实体匹配），仅依赖纯 HSH 阶段可能不够；此时可选的精排阶段必不可少。

### 8.4 对教师模型的依赖

我们基准中的真实邻居来自 bge-large 嵌入。因此，HSH-64 学习的是近似 bge-large 语义空间，而非某种客观的语义相似度概念。如果 bge-large 对某个中文歧义词表示错误，学到的哈希码也会继承该错误。这是所有基于蒸馏方法的共同局限。缓解策略包括使用多教师模型集成，或为歧义案例引入人工标注标签。

### 8.5 实际部署建议

对于具有严格延迟和内存预算的边缘设备，我们推荐 DH h256 模型（585 KB）配合 MIH 自适应搜索。它在纯 Hamming 空间中 Recall@10 超过 0.73，轻量精排后达到 0.8641。对于可使用更强精排器的服务端部署，四个模型集成达到 0.8970 Recall@10。

## 9 结论

本文提出了 HSH-64，一种面向轻量化语义检索的 64 位可学习语义哈希方案。HSH-64 结合了结构化编码格式、三阶段深度哈希训练流程、召回导向后处理、自适应 MIH 搜索以及可选的两阶段精排架构。在 3,109 词中文词表基准上，单个 585 KB 模型在纯 Hamming 空间中达到 0.7382 Recall@10，最佳后处理模型达到 0.7404，四模型集成达到 0.7724。在 MIH 粗排和 bge-large 精排下，系统达到 0.8970 Recall@10，同时保持在线编码器的小型化。这些结果表明，当训练与搜索流程经过协同设计时，紧凑的可学习哈希可以接近完整嵌入检索的准确率。

未来工作包括探索更大的位宽（如 128 位变体）、通过架构搜索增强集成多样性，以及将 HSH 码直接蒸馏到更小的学生模型以实现超低开销部署。

## References

- Bengio, Yoshua, Nicholas Léonard, and Aaron Courville (2013). “Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation”. In: *arXiv preprint arXiv:1308.3432*.
- Cao, Yue et al. (2015). “Deep multimodal hashing with orthogonal regularization”. In: *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 2291–2297.
- Courbariaux, Matthieu et al. (2016). “Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1”. In: *arXiv preprint arXiv:1602.02830*.

- Datar, Mayur et al. (2004). “Locality-sensitive hashing scheme based on p-stable distributions”. In: *Proceedings of the Twentieth Annual Symposium on Computational Geometry*, pp. 253–262.
- Devlin, Jacob et al. (2019). “BERT: Pre-training of deep bidirectional transformers for language understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 4171–4186.
- Guo, Jiafeng et al. (2020). “A deep look into neural ranking models for information retrieval”. In: *Information Processing & Management* 57.6, p. 102211.
- Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean (2015). “Distilling the knowledge in a neural network”. In: *Advances in Neural Information Processing Systems: Workshop on Deep Learning and Representation Learning*.
- Izacard, Gautier and Edouard Grave (2020). “Distilling knowledge from reader to retriever for question answering”. In: *arXiv preprint arXiv:2012.04584*.
- Jégou, Hervé, Matthijs Douze, and Cordelia Schmid (2011). “Product quantization for nearest neighbor search”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.1, pp. 117–128.
- Li, Wu-Jun, Sheng Wang, and Wang-Cheng Kang (2016). “Feature learning based deep supervised hashing with pairwise labels”. In: *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 1711–1717.
- Liu, Wei et al. (2012). “Supervised hashing with kernels”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2074–2081.
- Norouzi, Mohammad, Ali Punjani, and David J Fleet (2012). “Fast search in hamming space with multi-index hashing”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3108–3115.
- Reimers, Nils and Iryna Gurevych (2019). “Sentence-BERT: Sentence embeddings using Siamese BERT-networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pp. 3982–3992.
- Salakhutdinov, Ruslan and Geoffrey Hinton (2009). “Semantic hashing”. In: *International Journal of Approximate Reasoning* 50.7, pp. 969–978.
- Shen, Fumin et al. (2015). “Supervised discrete hashing”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 37–45.
- Wang, Jingdong et al. (2018). “A survey on learning to hash”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 40.4, pp. 769–790.
- Weiss, Yair, Antonio Torralba, and Rob Fergus (2008). “Spectral hashing”. In: *Advances in Neural Information Processing Systems*. Vol. 21, pp. 1753–1760.
- Xia, Rongkai et al. (2014). “Supervised hashing for image retrieval via image representation learning”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 28. 1.

## A 实现细节

### A.1 Deep Hash 模型架构

MLP 投影器的层维度如下：

- 输入：512 维 bge-small 嵌入，减去训练均值进行中心化。
- 隐藏层： $H$  个单元，ReLU 激活。
- 输出层：52 个线性单元（符号量化前的 logit）。

不使用批归一化、Dropout 或输出层偏置。总参数数为  $(512 + 1)H + (H + 1)52$ ，其中 +1 项为偏置。

### A.2 编码序列化格式

每个 HSH-64 模型以二进制文件存储，文件头如下：

- 魔数：4 字节（0xCAB1\_DE3D）。
- 版本：4 字节。
- 隐藏维度：4 字节。
- 输入维度：4 字节。
- 输出位数：4 字节。
- 均值向量： $D \times 4$  字节（float32）。
- 网络权重：剩余字节。

### A.3 MIH 索引构建

当  $B = 13$  段时，每段包含  $\lceil 52/13 \rceil = 4$  位，每段有  $2^4 = 16$  个可能值。每段的倒排索引将每个 4 位值映射到长度为  $N$  的位向量或有序项目标识符列表。查询处理时，枚举查询段在 Hamming 半径  $\lfloor r/B \rfloor$  内的所有值，收集候选并去重。

## B 复杂度分析

**编码成本。** 使用 bge-small + MLP 对查询编码需要一次  $512 \times H$  和一次  $H \times 52$  的矩阵乘法，即  $O(H(D + M))$  次运算。当  $H = 256$  时，约为  $512 \times 256 + 256 \times 52 \approx 144,000$  次乘法，在现代 CPU 上可忽略。

**暴力 Hamming 检索。** 将查询码与  $N$  个数据库码比较需要  $O(N)$  次 popcount 操作。当  $N = 3,109$  时，对小词表足够快，但无法扩展。

**MIH 检索。** 对于  $B$  段和查询半径  $r$ ，每段枚举  $V(4, \lfloor r/B \rfloor)$  个值。期望候选数为  $O(N \cdot V(M/B, r/B)/2^{M/B})$ 。在我们的实验中， $B = 13$  且自适应半径下，平均候选数为 187。

**内存成本。** 存储  $N$  个 64 位码需要  $8N$  字节（ $N = 3,109$  时约 25 KB）。MIH 索引约需  $B \cdot 2^{M/B} \cdot N$  位外加开销；当  $B = 13$  时，约为  $13 \times 16 \times 3,109$  位  $\approx 81$  KB。

## C 补充后处理网格搜索结果

表 6 列出了 h1024\_s2024 模型在 `max_iters=10` 时对  $P$  和  $w_{\text{neg}}$  的完整网格搜索结果。

表 6: h1024\_s2024 的完整后处理网格搜索。

| $P$ | neg_weight | Recall@10 | $\Delta$ |
|-----|------------|-----------|----------|
| 10  | 0.05       | 0.4169    | -0.0146  |
| 10  | 0.10       | 0.4296    | -0.0019  |
| 10  | 0.20       | 0.5044    | +0.0729  |
| 10  | 0.50       | 0.7194    | +0.2879  |
| 10  | 1.00       | 0.7280    | +0.2965  |
| 10  | 2.00       | 0.6798    | +0.2483  |
| 15  | 0.05       | 0.4098    | -0.0217  |
| 15  | 0.10       | 0.4118    | -0.0197  |
| 15  | 0.20       | 0.4235    | -0.0080  |
| 15  | 0.50       | 0.6255    | +0.1940  |
| 15  | 1.00       | 0.7229    | +0.2914  |
| 15  | 2.00       | 0.7106    | +0.2791  |
| 20  | 0.05       | 0.4097    | -0.0218  |
| 20  | 0.10       | 0.4114    | -0.0201  |
| 20  | 0.20       | 0.4231    | -0.0084  |
| 20  | 0.50       | 0.5860    | +0.1545  |
| 20  | 1.00       | 0.6922    | +0.2607  |
| 20  | 2.00       | 0.7140    | +0.2825  |

最佳配置为  $P = 10$ 、 $w_{\text{neg}} = 1.0$ ，Recall@10 为 0.7280。在所有配置中， $w_{\text{neg}}$  在  $[0.5, 2.0]$  范围内始终带来显著提升，而低于 0.2 的值无效甚至有害。